

Introduction à Python

Partie 2

TP Réseaux Avancés M1 SIC - IA

Ilyas Bambrik

Table des matières



I - Les chaînes de caractères	3
II - Exercice : Compter le nombre d'occurences	6
III - Exercice : Adresse IP	7
IV - Les classes et objets en python	8
V - Tuple et unpacking	11
VI - Exercice : Trie	13
VII - Fonctions prédéfinies	14
VIII - Paramètres de fonction par défaut	15
IX - Fonction lambda	17
X - Fonction d'ordre supérieur (higher order function)	18
XI - Exercice : Trie avec fonction d'ordre supérieur	20
XII - Exercice : Résumé Statistique Partie 2	22
XIII - Exercice : Server Load Balancing	25
XIV - Exercice : Surface Maximale	26
XV - Exercice : Maximiser l'entier	27

Les chaînes de caractères

I

Les chaînes de caractères sont représentées par le type *str* et fonctionnent d'une manière très similaire aux tableaux mais avec des méthodes supplémentaires.

La fonction *split()* (équivalente à la fonction *split* implémentée dans la classe *String* en Java) découpe une chaîne de caractères en morceaux selon le délimiteur (le caractère espace dans l'exemple suivant, *ligne 2*) et renvoie un tableau des éléments (sous-chaînes) selon le séparateur.

- Le code suivant divise les sous chaînes de *s* selon " " (*Ligne 2*). Le résultat renvoyé est placé dans la variable *Tableau*.
- Par défaut, la fonction *split* divise par " " si le séparateur n'est pas spécifié (*Ligne 4*).
- Dans *Ligne 7*, *split* divise par "sont".
- *Ligne 8* convertie la chaîne *s* en list (tableau)

```
1 s='Les chaines de caracteres sont représentees par le type str'
2 Tableau=s.split(" ")
3 print( "Tableau = ", Tableau)
4 Tableau=s.split()
5 print( "Tableau = ", Tableau)#
6 print( "s[10:20] = ", s[10:20])
7 print(s.split("sont"))
8 print(list(s))
```

Listing 9 Les objets str

```
Tableau = ['Les', 'chaines', 'de', 'caracteres', 'sont', 'représentees', 'p
ar', 'le', 'type', 'str']
Tableau = ['Les', 'chaines', 'de', 'caracteres', 'sont', 'représentees', 'p
ar', 'le', 'type', 'str']
s[10:20] = s de carac
['Les chaines de caracteres ', ' représentees par le type str']
['L', 'e', 's', ' ', 'c', 'h', 'a', 'i', 'n', 'e', 's', ' ', 'd', 'e', ' ',
'c', 'a', 'r', 'a', 'c', 't', 'e', 'r', 'e', 's', ' ', 's', 'o', 'n', 't', ' ',
', 'r', 'e', 'p', 'r', 'é', 's', 'e', 'n', 't', 'e', 'e', 's', ' ', 'p', 'a
', 'r', ' ', 'l', 'e', ' ', 't', 'y', 'p', 'e', ' ', 's', 't', 'r']
```

Les méthodes *str* suivantes sont accessibles :

- *lower()* : renvoie la chaîne de caractères en minuscule (*Listing 2*) ;
- *upper()* : renvoie la chaîne de caractères en majuscule (*Listing 3*);
- *title()* : renvoie la chaîne de caractères en title case (*Listing 4*);
- *find(substr)* : revoie la première position de la chaîne *substr* (*Listing 5*) dans la chaîne. Si celle-ci n'existe pas, -1 est retournée (*Listing 6*). Voir aussi *rfind(substr)* qui renvoie le dernier index au lieu du premier. Par ailleurs, *find(substr,start,end)* renvoie la position de la première occurrence entre *start* et *end* (sinon celle-ci renvoie -1);
- *count(substr)* : revoie le nombre d'occurrences de la chaîne *substr* (*Listing 7*).

- `substr in S` donne `True` si `substr` existe dans `S` (sensible à la case), `False` sinon.

```
1 s='Les chaines de caracteres sont representees par le type str'
2 print(s.lower())
3 print(s.upper())
4 print(s.title())
5 print(s.find("sont"))
6 print(s.find("####"))
7 print(s.count(" "))
8 print("sont" in s)
9 print("dans" in s)
```

```
les chaines de caracteres sont representees par le type str
LES CHAINES DE CARACTERES SONT REPRÉSENTEES PAR LE TYPE STR
Les Chaines De Caracteres Sont Représentées Par Le Type Str
26
-1
9
```

Pour ajuster une chaîne :

- `S.rjust(N,c)` : renvoie une chaîne de caractères de taille `N` constituée de $(N-\text{len}(s))*c$ à gauche et `S` à droite (*Ligne 1*);
- `S.ljust(N,c)` : renvoie une chaîne de caractères de taille `N` constituée de `S` à gauche et $(N-\text{len}(s))*c$ à droite (*Ligne 2*);
- `S.center(N,c)` : renvoie une chaîne de caractères de taille `N` avec `S` centrée au milieu (*Ligne 1*) ;

```
1 print("###".rjust(15, "?"))
2 print("###".ljust(15, "?"))
3 print("###".center(15, "?"))
```

```
1 ????????????###
2 ###????????????
3 ??????###??????
```

- `S.islower()` : retourne `True` si `S` est une chaîne minuscule. `False` sinon ;
- `S.isupper()` : retourne `True` si `S` est une chaîne majuscule. `False` sinon ;

D'autre méthodes similaires à tester : `isalnum()`, `isalpha()`, `isascii()`, `isdecimal()`, `isdigit()`, `isnumeric()`

- `S.startswith(prefix)` : retourne `True` si `S` commence par la sous chaîne *prefix*. `False` sinon ;
- `S.endswith(suffix)` : retourne `True` si `S` se termine par la sous chaîne *suffix*. `False` sinon ;
- `S.strip()` : renvoie une copie de `S` avec les espaces (" ", "\n" et "\t") du début et de la fin supprimés ;
- `S.lstrip()` : renvoie une copie de `S` avec les espaces (" ", "\n" et "\t") du début supprimés ;
- `S.rstrip()` : renvoie une copie de `S` avec les espaces (" ", "\n" et "\t") à la fin supprimés ;

Il est possible de spécifier les caractères à supprimer par *strip*, *lstrip* et *rstrip* (Ligne 6). Par exemple :

```
1 print("HELLO".isupper(), "HELLO".islower())
2 print("hello".isupper(), "hello".islower())
3 print("Hello".isupper(), "Hello".islower())
4 print(" \n \t text text text \t\t\n".strip())
5 print("ABC ".rstrip())
6 print("####?ABC##?##".strip("#")) # supprimer # du début/fin
```

```
1 True False
2 False True
```

```
3 False False
4 text text text
5 ABC
6 ??ABC##???
```

La méthode `sep.join(list_str)` prends une liste de chaînes de caractères (*list_str*) et colle les chaînes de celle-ci avec le séparateur (*sep*).

```
1 print(" + ".join(["23", "13", "5"]))
2 # resultat est '23 + 13 + 5'
```

Exercice : Compter le nombre d'occurrences



Question

Dans ce défi, l'utilisateur saisit une chaîne et une sous-chaîne. Vous devez imprimer le nombre de fois que la sous-chaîne apparaît dans la chaîne donnée.

REMARQUE : les lettres de la chaîne sont sensibles à la casse.

Input

La première ligne d'entrée contient la chaîne d'origine. La ligne suivante contient la sous-chaîne.

Output

Affiche le nombre entier indiquant le nombre total d'occurrences de la sous-chaîne dans la chaîne d'origine.

Exemple input

ABCD CDC

CD

Résultat

2

Indice :

$A.count(B)$ retourne le nombre de fois la chaîne de caractères B se répète dans A .

Exercice : Adresse IP



Question

Écrivez une fonction qui prend en paramètre une adresse IP sous forme de chaînes de caractères et retourne la transformation de celle-ci sous forme d'une suite de 32 bites.

Exemple :

Pour l'adresse IP "91.129.3.55" en entrée le résultat sera :

91 = '01011011'

129 = '10000001'

3 = '00000011'

55 = '00110111'

Le résultat finale est :

01011011100000010000001100110111

Indice :

- Utilisez *split* pour récupérer les valeurs des 4 octets de l'adresse IP ;
- Avant de transformer la chaîne représentant l'octet en valeur binaire, transformez la en entier (avec l'appel de *int*) ;
- Utilisez *rjust* pour ajuster la taille du résultat de la fonction *bin* (il faudra aussi supprimer le préfixe '0b' du résultat de *bin* avec un *slicer*) ;
- Utiliser *join* pour concaténer le résultat des 4 octets ou bien utilisez la concaténation ordinaire avec l'opérateur +;

Les classes et objets en python

IV

Python supporte de même les concepts de la POO. Le programme illustré dans *Listing III.6* présente une classe *TypeJoueur* :

Listing 10 Les classes et objets en python

```

1 class TypeJoueur:
2     def __init__(self):
3         self.PointsDeVie=100
4         self.Score=0
5         self.NomJoueur=' '
6
7     def lireNomJoueur(self):
8         self.NomJoueur = input()
9
10    def mettreAJourScore(self,scoreRecompense):
11        self.Score=self.Score+scoreRecompense
12
13    def afficherStatisticJoueur(self):
14        print( "NomJoueur=",self.NomJoueur,"\nPoints De Vie=",self.PointsDeVie,
15              "\nScore=",self.Score)
16 Med=TypeJoueur()
17
18 Med.lireNomJoueur()
19 Med.mettreAJourScore(10)
20 Med.afficherStatisticJoueur()

```

- La méthode `def __init__(self):` représente le constructeur de la classe en python.
- Le mot clé `self`, passé en argument dans tous les déclarations de méthode de la classe *TypeJoueur*, représente l'objet qui a invoqué la méthode ou le constructeur. Cependant, lors de l'appel d'une méthode, cet objet `self` est automatiquement inféré. Par exemple :
`Med=TypeJoueur()` # est équivalente à `Med=TypeJoueur(Med)`. Donc l'objet courant (`self`) prendra la référence de l'objet qui a invoqué la méthode
`Med.mettreAJourScore(10)` # de même cette instruction est équivalente à `Med.mettreAJourScore(Med, 10)`
- En outre, les attributs de la classe sont déclarés à l'intérieur du constructeur `__init__` (`self.PointsDeVie`, `self.Score`, `self.NomJoueur`).
- L'appel du constructeur (*ligne 16*) se fait sans opérateur *new* contrairement à la syntaxe Java.

Exemple : Constructeur paramétré

Le code suivant définit un constructeur avec les arguments *pointsvie*, *score* et *nom* :

Listing 11 Déclaration d'un constructeur paramétré en python

```
1 def __init__(self,pointsvie,score,nom):
2     self.PointsDeVie=pointsvie
3     self.Score=score
4     self.NomJoueur=nom
```

Il est possible aussi de définir l'implémentation des opérateurs comme +, -, *, [], (). Par exemple, la classe *list* (tableau) définit l'opérateur + pour concaténer une liste avec une autre et * pour répéter une liste un certain nombre de fois. Voir l'exemple suivant :

```
1 A=[10,20,30,40]
2 B=[100,200,300,400]
3 C=A+B
4 print(C) # [10, 20, 30, 40, 100, 200, 300, 400]
5 C=A*3
6 print(C) # [10, 20, 30, 40, 10, 20, 30, 40, 10, 20, 30, 40]
```

La surcharge d'opérateur peut être définie dans une classe avec la méthode correspondante à l'opérateur spécifique. Par exemple, + est définie par la méthode `__add__`, * par `__mul__`, / par `__div__`. L'exemple suivant définit l'opérateur d'addition (`__add__` ligne 15) pour modifier le score du joueur et l'opérateur de multiplication (`__mul__` ligne 18) pour multiplier le score par un facteur. Les deux méthodes précédentes retournent l'objet après la modification du score (lignes 17 et 20). La surcharge des deux opérateurs est testée dans lignes 26 et 28.

```
1 class TypeJoueur:
2     def __init__(self):
3         self.PointsDeVie=100
4         self.Score=0
5         self.NomJoueur=''
6
7     def lireNomJoueur(self):
8         self.NomJoueur = input()
9
10    def mettreAJourScore(self,scoreRecompense):
11        self.Score=self.Score+scoreRecompense
12
13    def afficherStatisticJoueur(self):
14        print( "NomJoueur=",self.NomJoueur,"\nPoints De Vie=",self.PointsDeVie,
15              "\nScore=",self.Score)
16    def __add__(self,scoreRecompense):
17        self.Score=self.Score+scoreRecompense
18        return self
19    def __mul__(self,multiplicateurScore):
20        self.Score=self.Score*multiplicateurScore
21        return self
22
23 Med=TypeJoueur()
24 Med.lireNomJoueur()
25 Med.mettreAJourScore(10)
26 Med+=20
27 Med.afficherStatisticJoueur()
```

```
28 Med*=5  
29 Med.afficherStatisticJoueur()
```

Tuple et unpacking



Python offre un autre type similaire au tableau appelé tuple. Les tuples supportent la plus part des opérations sur les tableaux sauf qu'ils sont immuables (une fois que le tuple est créé, on ne peut pas changer son contenu).

```
>>> Etudiant=("Mohammed",18,"60kg","L1")
>>> print Etudiant
('Mohammed', 18, '60kg', 'L1')
>>> print type(Etudiant)
<type 'tuple'>
>>> print Etudiant[1]
18
>>> for attribut in Etudiant:
    print attribut,

Mohammed 18 60kg L1
>>> Etudiant[1]=20

Traceback (most recent call last):
  File "<pyshell#7>", line 1, in <module>
    Etudiant[1]=20
TypeError: 'tuple' object does not support item assignment
```

Les tableaux et les tuples sont souvent utilisés pour débiller (unpack) plusieurs valeurs. Supposant que ont possède un tuple T contenant le nom, l'age et le niveau d'un étudiant et on souhaite attribuer chaque valeur à une variable.

```
1 T= ("Mohammed", 18, "L3")
2 nom, age, niveau=T
3 print (nom)
4 print (age)
5 print (niveau)
```

- Le tuple précédant peut être écrit sans les parenthèses :
T="Mohammed",18,"L3"
- La même syntaxe s'applique avec les tableaux. Ceci est utile pour retourner plusieurs valeurs au même temps par une fonction.

Syntaxe

L'instruction "nom,age,niveau=T" est équivalente à :

nom=T[0]

age=T[1]

niveau=T[2]

Cette syntaxe peut être utilisée afin de permuter les valeurs des variables :

`a=1`

`b=2`

`a, b=b, a`

ou bien

`a, b=[b, a]`

Permet de placer la valeur de a dans b et la valeur de b dans a.

Exercice : Trie

VI

Question

Le code suivant lit à partir du clavier la taille du tableau (*Ligne 2*) et remplit celui-ci avec des valeurs aléatoires (*Ligne 3*).

Complétez le code (*Ligne 6 et 7*) pour effectuer le trie ascendant du tableau A

```
1 from random import random
2 N=int(input())
3 A=[int(random()*10000) for i in range(N)]
4 for i in range(N):
5     for j in range(i,N):
6         # Comparez A[i] et A[j]
7         # Si A[i]>A[j], permutez les deux valeurs
8 print(A)
9
```

Fonctions prédéfinies

VII

Python propose un beaucoup de fonctions et modules prédéfinies :

- `ord(chr)` : prend en paramètre un caractère et renvoie son code ascii ;
- `chr(code_ascii)` : rend un code ascii et renvoie le caractère ascii correspondant ;
- `sum()` : prend en paramètre un tableau de valeurs numériques et renvoie la somme de ce dernier ;
- `min()` : prend en paramètre un tableau de valeurs numériques et renvoie le minimum ;
- `max()` : prend en paramètre un tableau de valeurs numériques et renvoie le maximum;
- `bin(entier)` : prend en paramètre un entier et renvoie ça représentation binaire sous forme de chaîne de caractères;

```
1 print(ord('A'))
2 print(chr(65))
3 print(sum([24, 53, 32, 212]))
4 print(min([24, 53, 32, 212]))
5 print(max([24, 53, 32, 212]))
6 print(min(24, 53, 32, 212))
7 print(max(24, 53, 32, 212))
8 print(bin(32346))
```

```
1 65
2 A
3 321
4 24
5 212
6 24
7 212
8 0b111111001011010
```

Le module *re* permet d'utiliser les expressions régulières :

```
1 import re
2 text='Les chaines de caracteres, sont representees par le type str'
3 print(dir(re))
4 print(re.findall("^[^s]+",text))
5 print(re.split("[^s]+",text))
```

Paramètres de fonction par défaut

VIII

Dans une fonction, il est possible de déclarer un ou plusieurs paramètres par défaut.

```
1 def multiplier(nombre, facteur=2):
2     return nombre*facteur
3 print( multiplier(3,5)) # resultat =15
4 print( multiplier(7)) # resultat = 14
```

- Dans l'exemple précédent, la fonction multiplier possède *un paramètre par défaut facteur avec une valeur 2*. Ceci dit que si la valeur du paramètre n'est pas précisée, la valeur de ce dernier est égale à 2 (voir l'appel de multiplier avec un seul paramètre).
- Il est possible de déclarer plusieurs paramètres par défaut. La seule restriction est que les paramètres par défaut ne sont pas suivies par un paramètre obligatoire :

L'écriture suivante est correcte :

```
def fonction(argobligatoire1,argobligatoire2,parametrepardefaut1=3,parametrepardefaut2=2,
parametrepardefaut3=1) ;
```

Par contre l'écriture suivante est incorrecte car *argobligatoire2* vient après *parametrepardefaut3* ;

```
def fonction(argobligatoire1,parametrepardefaut1=3,parametrepardefaut2=2,parametrepardefaut3=1,
argobligatoire2)
```

Remarque : Variable globale

A l'intérieur d'une fonction, pour avoir accès à une ou plusieurs variables globales, il est nécessaire d'utiliser le mot clé *global* suivi par les variables globales séparées par des virgules :

```
1 variableGlobal=4
2 def impromerlavaleurdeVG():
3     global variableGlobal
4     print( variableGlobal)
5 print( impromerlavaleurdeVG())
```

Il est possible de définir un nombre d'arguments variable pour une fonction et de récupérer ces derniers dans un tuple dans le corps de la fonction :

- l'argument qui doit contenir un nombre variable de paramètres est déclaré avec un *** comme préfixe (*Ligne 1*) ;
- Si la fonction est définie avec des arguments obligatoires ainsi que une liste d'arguments variables, l'argument contenant les arguments variables doit obligatoirement être définie à la fin de la signature de la fonction (*Ligne 8*);

- Cependant, il est possible d'avoir des arguments optionnels après la liste d'arguments de taille variable (*Ligne 11*). Cependant lors de l'appel de la fonction, il est nécessaire de spécifier les noms des arguments lors de l'appel (*Ligne 13*) . Sinon, l'argument optionnel sera pris comme l'une des valeur de la liste d'arguments de taille variable (*Ligne 14*);

```

1 def fonction1(*arguments):
2     print("arguments de la fonction1=",arguments)
3 fonction1(1,2,3)
4 fonction1(5)
5 fonction1(7,8,9,10)
6 fonction1()
7 fonction1(*list(range(10)))
8 def fonction2(arg1,arg2,arg3,*arguments):
9     print("arguments de la fonction2=",arguments)
10 fonction2(7,8,9,10,11)
11 def fonction3(arg1,arg2,arg3,*arguments,param_opt=1):
12     print("arguments de la fonction3=",arguments," ",param_opt)
13 fonction3(7,8,9,10,11,param_opt=23)
14 fonction3(7,8,9,10,11,23)

```

```

1 arguments de la fonction1= (1, 2, 3)
2 arguments de la fonction1= (5,)
3 arguments de la fonction1= (7, 8, 9, 10)
4 arguments de la fonction1= ()
5 arguments de la fonction1= (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
6 arguments de la fonction2= (10, 11)
7 arguments de la fonction3= (10, 11) , 23
8 arguments de la fonction3= (10, 11, 23) , 1

```

Similairement à la liste d'arguments de taille variable il est possible de définir une liste d'arguments de taille variable nominés. Dans le corps de la fonction, les arguments nominés sont accessibles sous forme d'un dictionnaire.

```

1 def fonction4(arg1,arg2,arg3,**options):
2     print("arguments nominee de la fonction4=",options)
3 fonction4(7,8,9,argument1=10,argument2=11)

```

```

1 arguments nominee de la fonction4= {'argument1': 10, 'argument2': 11}

```


Fonction lambda

IX

Souvent, il est nécessaire de déclarer des fonctions qui assurent un traitement limité ou calculer une formule prédéfinie.

Par exemple supposant qu'on souhaite écrire une fonction permettant de convertir la température en degré Celsius vers Fahrenheit.

$\text{temp_Fahr} = (\text{temp_celsius} \times 9/5) + 32$

```
1 def convertirCelsiusToFahr(temp_celsius):
2     return (temp_celsius * 9./5) + 32
```

A ce niveau, il faudra noter qu'une fonction peut être affectée à une variable. Une fonction déclarée en python n'est qu'un objet de type *function* :

- Dans l'exemple suivant on affecte à la variable *var1* la fonction *convertirCelsiusToFahr*;
- Par la suite, dans *ligne 3* avec la fonction *type* il est possible de voir que le type de la variable *var1* est "<type 'function'>" ;
- Il est même possible de faire appel à la procédure stockée dans *var1* avec *var1(val)* (*Ligne 5*) ;
- Ainsi les fonctions sont appelées *first class citizens* car ils sont traitées comme des variables / objets et peuvent être utilisées comme des arguments pour d'autres fonctions.

```
1 def convertirCelsiusToFahr(temp_celsius):
2     return (temp_celsius * 9./5) + 32
3 var1=convertirCelsiusToFahr
4 print (type(var1)) # imprime <type 'function'>
5 print ( var1(3.))
```

Comme d'autres langages évolués, il existe une façon plus concise afin de déclarer une telle opération appelée *expression lambda*:

```
1 fun=lambda temp_celsius:(temp_celsius * 9./5) + 32
2 print( fun(3.))
```

- L'expression lambda précédente déclare une fonction anonyme avec un seul paramètre *temp_celsius*, et retourne la valeur de la formule $(\text{temp_celsius} * 9/5) + 32$.
- La fonction déclarée est attribuée à la variable *fun*. Par la suite, afin d'invoquer cette fonction il suffit d'utiliser la variable *fun* (*Ligne 2*).
- Dans cette déclaration le mot clé *return* n'est pas utilisé alors que la fonction retourne la valeur de la formule. Ceci est appelé *implicite return*.

Fonction d'ordre supérieur (higher order function)



Une *fonction d'ordre supérieur* est une fonction qui prend en paramètre une/plusieurs autre(s) fonction(s) ou retourne une fonction. Il existe plusieurs fonctions d'ordre supérieurs prédéfinies qui sont très utiles.

👉 Exemple : map

Supposant qu'on souhaite lire une liste d'entiers, séparés par des espaces, à partir du clavier. Ce travail peut être accompli de la manière suivante :

```
1 def convertirEnListEntier(L):
2     resultat=[]
3     for valeur in L:
4         resultat.append(int(valeur))
5     return resultat
6 print( convertirEnListEntier(input().split(" ")) )
```

Le programme précédant prend chaque valeur de la liste, la convertit vers le type *int* en faisant appel à *int(valeur)* et ajoute le résultat au tableau *resultat*. Il existe une fonction qui permet d'appliquer une fonction à chaque élément de la liste et de retourner la liste contenant le résultat sans changer la liste originale. Cette fonction est appelée *map* :

```
1 resultat = list(map(int, input().split(" ")))
2 print( resultat)
```

- La fonction *map*, prend une fonction en paramètre (dans cet exemple *int*) et une collection / tableau, applique cette fonction à chaque élément et ajoute le résultat à la collection en sortie.
- L'avantage de ceci est que si on souhaite changer le type de conversion ou la fonction appliquée sur les éléments de la liste, il suffit de changer la fonction argument de *map*. Par exemple supposant que on souhaite avoir les valeurs de la liste saisie du clavier comme des doubles (*float*). La seule chose qui a changée est la fonction passée en paramètre (*float au lieu de int*).

```
1 resultat = list(map(float, input().split(" ")))
2 print( resultat)
```

👉 Exemple : filter

Une autre fonction similaire à *map*, est la fonction *filter*. Supposant qu'on possède une liste des notes et on souhaite retenir seulement les notes supérieures à 14. La fonction suivante fait exactement ça :

```
1 def notesSupA14(L):
```

```

2     resultat=[]
3     for valeur in L:
4         if valeur>14:
5             resultat.append(int(valeur))
6     return resultat
7 Liste=[13, 15, 16, 10, 9, 18, 11, 10]
8 print( notesSupA14(Liste))

```

Par contre, avec *filter*, il est plus facile d'accomplir ce traitement :

```

1 def supp14(note):
2     return note>14
3 print(list(filter(supp14,Liste)))

```

- Il suffit de définir une fonction qui va retourner une valeur booléenne (True ou False) et la mettre comme paramètre pour filter. Appelons cette fonction comme la fonction d'évaluation.
- La fonction d'évaluation va être appliquée sur chaque élément de la liste. L'élément est ajouté à la collection résultat si l'évaluation retourne *True*.
- Il est possible d'écrire la fonction précédente d'une manière plus concise avec une expression lambda :

```

1 Liste=[13, 15, 16, 10, 9, 18, 11, 10]
2 print filter(lambda valeur:valeur>14,Liste)

```

Exercice : Trie avec fonction d'ordre supérieur

XI

Soit la liste python des étudiants où chaque étudiants est représenté par un dictionnaire contenant les attributs (nom, identifiant et note):

```
1 etudiants=[
2   {
3     "nom": "Dupont",
4     "identifiant": "ETU020",
5     "note": 15.5
6   },
7   {
8     "nom": "Martin",
9     "identifiant": "ETU002",
10    "note": 18
11  },
12  {
13    "nom": "Petit",
14    "identifiant": "ETU013",
15    "note": 12.75
16  },
17  {
18    "nom": "Lefebvre",
19    "identifiant": "ETU003",
20    "note": 9
21  },
22  {
23    "nom": "Richard",
24    "identifiant": "ETU001",
25    "note": 16.5
26  }
27 ]
```

Question 1

Complétez le code suivant afin de trier la liste des étudiants en paramètre dans un ordre ascendant. La fonction *trie_etudiants* prend deux arguments :

- *Liste_etudiants* qui contient la liste des étudiants sous forme JSON ;
- *function_comp* qui est une fonction qui prend en paramètre *Liste_etudiants[i]* et *Liste_etudiants[j]* et retourne *True* ou *False* selon le résultat de la comparaison ;

Remplacez la valeur *True* (Ligne 5) par l'appel de *function_comp* sur le *i*em et *j*em elements de *Liste_etudiants* ;

```
1 def trie_etudiants(liste_etudiants, function_comp):
```

```

2     N=len(liste_etudiants)
3     for i in range(N):
4         for j in range(i+1,N):
5             if True:
6                 liste_etudiants[i],liste_etudiants[j]=liste_etudiants[j],
liste_etudiants[i]

```

Question 2

Exécutez deux appels de la fonction *trie_etudiants* avec comme paramètres la liste des étudiants JSON et :

- Pour le premier appel, une fonction prenant deux paramètres (dictionnaires représentant les deux étudiants à comparer) et qui retourne *True* si le premier étudiant possède une meilleure note que la note du deuxième. (Trie ascendant selon "*note*") ;
- Pour le deuxième appel, une fonction prenant deux paramètres (dictionnaires représentant les deux étudiants à comparer) et qui retourne *True* si le premier étudiant possède un nom alphabétiquement supérieur que le nom du deuxième. (Trie ascendant selon "*nom*") ;

Pour chaque appel, imprimer la liste des étudiants pour vérifier que la liste est triée proprement.

Question 3

- Triez la liste des étudiants avec la méthode prédéfinie *sort()* et le paramètre optionnel *key*. Le paramètre optionnel *key* prend en paramètre une fonction qui retourne la valeur à comparer lors du trie ;
- Testez le trie selon la clé identifiant et vérifiez que le tableau est bien trié ;

Exercice : Résumé Statistique Partie 2

XII

Exercice sur les tableaux et boucles

Cet exercice consiste à refaire Résumé Statistique Partie 1 mais en utilisant les fonction d'ordre supérieure, les fonctions prédéfinies de python (map,filter, sort , sorted, sum, etc) et dictionnaire pour résoudre les mêmes problèmes d'une manière plus concise.

```

1 Notes=[15, 13, 5, 11, 5, 14, 9, 6, 14, 1,
2         8, 8, 11, 3, 12, 11, 2, 17, 14, 1, 6, 12, 13, 14, 1]
3 # Question 1 convertir les elements du tableau Notes du type en entier (int) au
   type float.
4 # et imprimer le resultat sur ecran
5 """ le resultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0,
   1.0, 8.0, 8.0, 11.0,
6 3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]"""
7
8
9
10
11 # Question 2 complete le programme pour calculer et imprimer la moyenne sur
   l'ecran
12 # Indication : utiliser le tableau apres conversion des element en float
13 # le resultat attendu = 9.04
14
15
16
17 # Question 3 calculez la variance type de Notes et imprimer le resultat sur ecran
18 # le resultat attendu = 23.6384
19
20
21
22 # Question 4 calculez la valeur la plus frequente dans le tableau et imprimez le
   resultat sur ecran
23 # le resultat attendu = 14.0 ou 14 (se repete 4 fois)
24
25
26 # Question 5 triez le tableau Notes sans utiliser les methodes predefinies ( .
   sort() et sorted ) dans l'ordre ascendant
27 """le resultat attendu
28 [1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0,
29 11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]
30 ou bien
31 [1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11,
32 11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]
33 """
34
35 # Question 6 creez une liste contenant seulement les valeurs uniques de Notes:

```

```

36 # Indication : utilisez le tableau apres le trie ou l'operateur 'in' pour tester
37 # l'appartenance
38 # resultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]
39
40
41

```

Question 1

Convertir les éléments du tableau Notes du type en entier (int) au type float et imprimer le résultat sur écran.

Indice :

Le résultat attendu = [15.0, 13.0, 5.0, 11.0, 5.0, 14.0, 9.0, 6.0, 14.0, 1.0, 8.0, 8.0, 11.0, 3.0, 12.0, 11.0, 2.0, 17.0, 14.0, 1.0, 6.0, 12.0, 13.0, 14.0, 1.0]

Question 2

Compléter le programme pour calculer et imprimer la moyenne sur l'écran.

Indice :

Indication : utiliser le tableau après conversion des élément en float

Le résultat attendu = 9.04

Question 3

Calculez la variance de Notes et imprimer le résultat sur écran. ($E(X)$ n'est que la moyenne que vous venez de calculer dans la question précédente).

$$\text{Var}(X) = \frac{\sum_{i=1}^n (X_i - E(X))^2}{n} = E((X - E(X))^2)$$

Indice :

Le résultat attendu = 23.6384

Question 4

Calculer la valeur la plus fréquente dans le tableau et imprimez le résultat sur écran.

Indice :

Le résultat attendu = 14.0 ou 14 (se répète 4 fois)

Question 5

Triez le tableau Notes sans utiliser les méthodes prédéfinies (`.sort()` et `sorted`) dans l'ordre ascendant.

Indice :

Le résultat attendu

[1.0, 1.0, 1.0, 2.0, 3.0, 5.0, 5.0, 6.0, 6.0, 8.0, 8.0, 9.0, 11.0,

11.0, 11.0, 12.0, 12.0, 13.0, 13.0, 14.0, 14.0, 14.0, 14.0, 15.0, 17.0]

ou bien

[1, 1, 1, 2, 3, 5, 5, 6, 6, 8, 8, 9, 11,

11, 11, 12, 12, 13, 13, 14, 14, 14, 14, 15, 17]

Question 6

Créez une liste contenant seulement les valeurs uniques de Notes.

Indice :

Indication : utilisez le tableau après le trie ou l'opérateur 'in' pour tester l'appartenance.

Résultat attendu =[1, 2, 3, 5, 6, 8, 9, 11, 12, 13, 14, 15, 17]

Exercice : Server Load Balancing

XIII

Question

Dans ce défi, on vous donne N serveurs. Chaque serveur dispose d'un courant de charge L_i de tâches en cours d'exécution. Ensuite un batch de k tâches arrive et doit être distribué sur les N serveurs. Votre travail consiste à concevoir un programme qui distribuera les travaux k entrants sur les serveurs de sorte que la différence entre le nombre de travaux sur le serveur avec la charge la plus élevée et celui avec la charge la plus faible soit minimale. Appelons cette métrique Déséquilibre minimal.

Par exemple si nous avons $N=4$ et les charges initiales comme suit $[5,0,2,1]$. Pour $k=3$ tâches arrivantes, la distribution des travaux pour obtenir ce qui suit $[5,2,2,2]$ permet d'obtenir le déséquilibre minimal. Le résultat ici est 3 (5-2)

Ce défi était l'un des problèmes d'Amazon Last Mile 22.

Input :

Ligne 1 : N Un entier indiquant le nombre de serveurs

Ligne 2 : k Un entier indiquant le nombre de travaux à planifier

Ligne 3 : Une ligne contenant des entiers L_i où chaque entier indique la charge actuelle du serveur

Output :

Un entier indiquant le déséquilibre minimum réalisable après la planification des k tâches.

Contraintes :

$1 \leq N < 10000$

$0 \leq L_i \leq 10000$

$0 \leq k \leq 100000000$

Exemple :

6

5

5 8 7 1 4 3

Sortie :

4

Exercice : Surface Maximale

XIV

Question

Bob joue à LEGO. Il a N petites briques. Chaque brique est un cube $1 \times 1 \times 1$.

En utilisant TOUTES les briques, Bob construit un gros "bloc". Ce bloc doit être un cuboïde simple (c'est-à-dire sans espace vide à l'intérieur). Bob s'intéresse à la surface externe du bloc (somme des surfaces des 6 faces) qu'il a construit.

Quelles sont les surfaces minimales et maximales possibles que Bob peut construire avec les N briques?

Input :

Un entier N , le nombre de petites briques

Output :

Une ligne, avec deux nombres entiers séparés par un espace.

Le premier entier est la surface minimale.

Le deuxième entier est la surface maximale.

Contraintes :

$1 \leq N \leq 1\,500\,000$

Exemple 1:

1

Sortie 1:

6 6

Exemple 2:

9

Sortie 2 :

30 38

Exemple 3 :

144

Sortie 2 :

168 578

Exercice : Maximiser l'entier

XV

Question

On vous donne un entier N. Votre but est de créer le plus grand entier possible.

Pour ce faire, vous pouvez échanger deux chiffres adjacents tant qu'ils sont de parité différente. Par exemple, vous pouvez échanger un chiffre impair et un chiffre pair, mais pas deux chiffres pairs. Vous pouvez effectuer autant d'échanges que vous le souhaitez.

Exemple :

Input :

Un entier N.

Output :

L'entier maximisé.

Contraintes :

N est composé d'au plus 900 chiffres.

Exemple :

325

Sortie :

352